- ***Chai*—** Chai provides assertion matchers to help make your tests more readable. Find out more at http://chaijs.com/ or `npm install chai`.
- ***Zombie*—** Zombie provides a virtual browser (a mock browser) that you can use inside Node.js. Find out more at http://zombie.labnotes.org/ or `npm install zombie`. *Zombie is written in CoffeeScript*.

Modules

In chapter 12 you created your own support for Node.js module syntax in your browser-based program. There are a few alternatives to look at while you wait for JavaScript runtimes to support some native module system:

- ***Stitch*—** Stitch uses a setup similar to the one you built in chapter 12. Find out more at https://github.com/sstephenson/stitch or `npm install stitch`. *Stitch is written in CoffeeScript*.
- ***Browserify*—** Browserify provides a compiler to support Node.js modules. Find out more at http://browserify.org/ or `npm install gamma`.
- ***Bower*—** Bower provides a complete package system for your browser-based programs, similar to some of the features that npm provides on Node.js. Find out more at http://bower.io/.

Builds

In chapter 12 you used Cake to build your program. Although useful, Cake can be overly minimal for some needs:

- ***Make*—** Make is not specific to CoffeeScript, JavaScript, or Node.js, but it's one of the most widely used build automation tools. Although Make can be intimidating to novices, it's an important and effective tool that you should learn to use. Find out more at http://www.gnu.org/software/make/.